Werkproces uitwerking SD-Portfolio

Werkproces	K1W3: Realiseert (onderdelen van) software
Studentnaam	Yakub Bulut
Studentnummer	97118569
Klascode	SDA2
Datum oplevering	9/28/2026

Student verklaart hierbij dat de bewijsstukken in dit STARR-document

- Authentiek zijn
- Kwalitatief goed zijn
- Binnen een redelijke tijdsduur tot stand zijn gekomen

Bewijsstuk

Per bewijsstuk moet er gekeken worden naar hoe sterk deze is, hiervoor kijk je naar twee dingen:

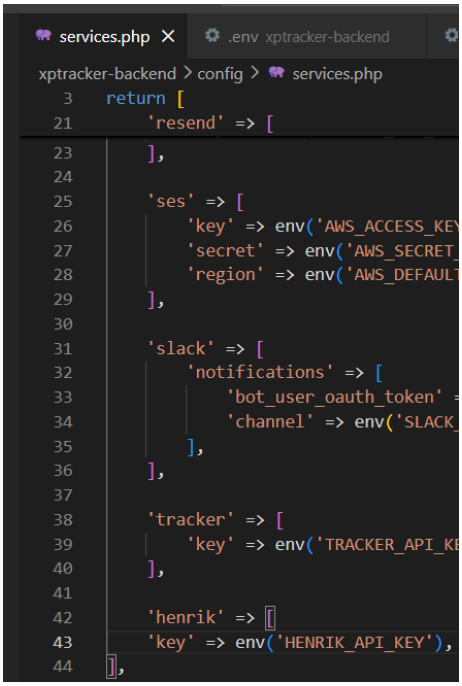
1. De context – Waar en met wie heb je het gedaan?
2. Je zelfstandigheid– Hoe zelfstandig heb je gewerkt?

Voor zowel de context als de zelfstandigheid kan een score A (hoog), B (middelmatig) of C (laag) gegeven worden. Een bewijsstuk met de score ‘A - A’ (goede context én zelfstandig gewerkt) heeft een hogere waarde dan een bewijsstuk met de score ‘B - C’.

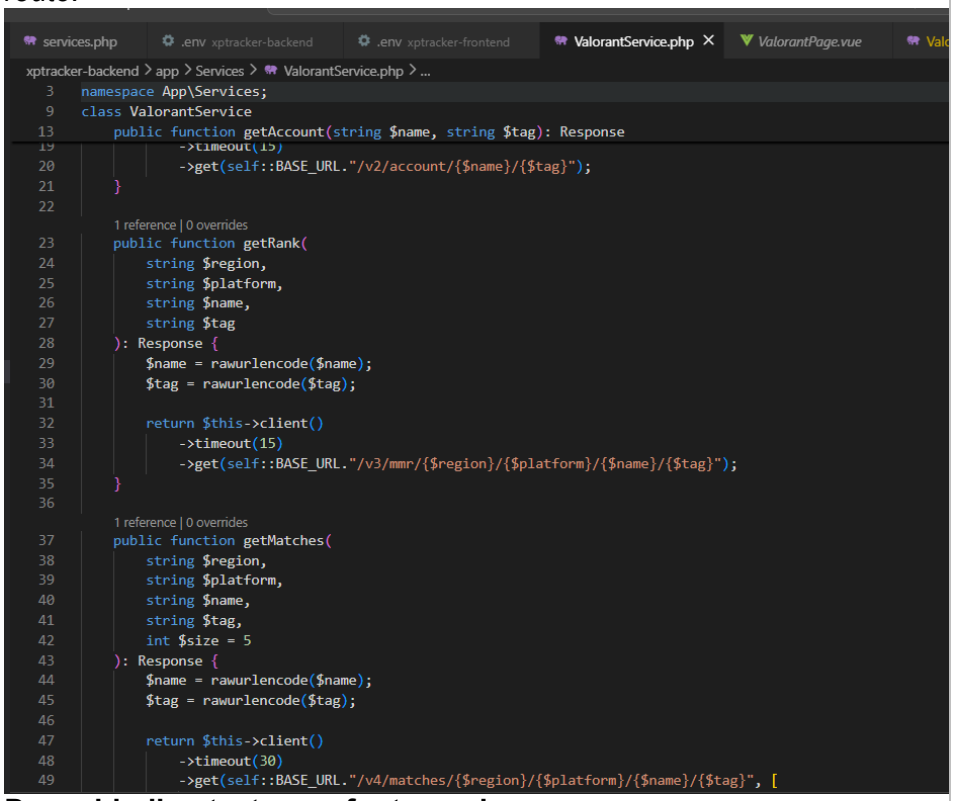
Maak voor het bepalen van de complexiteit gebruik van de "Context check" en de "Zelfstandigheid check”.

Korte beschrijving	Context	Zelfstandigheid
In deze bewijsstuk laat ik zien hoe ik een api connect aan mijn front-end en back-end en hoe ik het realiseer ook github en azuredevops	alleen Context check Uitkomst (B)	Context check Uitkomst (C)

STARR-reflectie

Situatie	**Ik heb dit tijdens de lessen op school gemaakt in september 2026. We werkten met ons groepje aan XPTracker. Mijn taak was om de Valorant API te koppelen, zodat je spelers kunt opzoeken en hun rank en punten kunt bekijken. Ik werkte aan de backend en de koppeling met de frontend. Mijn code hield ik bij op GitHub met commits, branches en merges.**
Taak	tijdens de lessen aan de Valorant koppeling van XPTracker. Dit was mijn onderdeel binnen het werkproces 'Realiseert software'. 1. Ik heb de Valorant API aan onze Laravel-backend gekoppeld, zodat we de rank en punten van een speler kunnen ophalen. 2. Ik heb de frontend gekoppeld aan de backend, zodat je een speler kunt zoeken en zijn gegevens op de pagina ziet. 3. Ik heb controles en foutmeldingen toegevoegd, zodat de gebruiker een duidelijke melding krijgt als een speler niet gevonden wordt of de API niet bereikbaar is.
Activiteiten	1. **De verbinding instellen** Ik begon met het instellen van de HenrikDev API in de Laravel-backend. De API-sleutel zette ik in .env. In config/services.php maakte ik een verwijzing naar die sleutel, zodat deze niet rechtstreeks in mijn code staat. *Afbeelding 1: de instellingen in config/services.php, zonder de echte API-sleutel.*  2. **De backend opbouwen** Ik maakte ValorantService.php voor het ophalen van de gegevens. Daarna maakte ik in de controller de verwerking van de aanvraag en voegde ik de routes toe. Door dit te verdelen over verschillende bestanden is duidelijk waar welk onderdeel staat. *Afbeelding 2: de methode getRank() in de service en de bijbehorende*

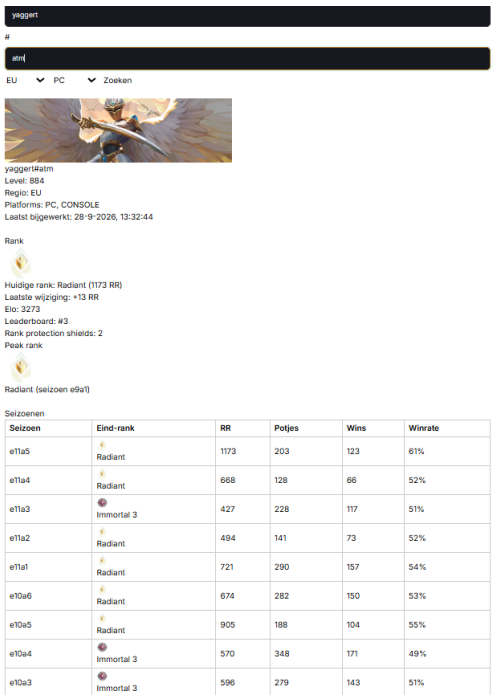
route.



3. **De verbinding testen en fouten oplossen**
   Ik testte de API in de browser met de speler yaggert#atm. Eerst kon Laravel de service niet vinden. Nadat dit was opgelost, kreeg ik een foutmelding over de API-sleutel. Met een controle in Tinker zag ik dat Laravel de sleutel niet inlas. Ik paste de configuratie aan en testte opnieuw. Daarna kwamen de spelersgegevens terug.
   Afbeelding 3: de foutmelding bij het ophalen van de gegevens.



Afbeelding 4: het JSON-antwoord met de opgehaalde spelersgegevens.



4. **De frontend koppelen**
   Daarna werkte ik aan de frontend, zodat een gebruiker een speler kan opzoeken en de gegevens op de pagina kan bekijken. De frontend vraagt de gegevens op bij onze Laravel-backend. De geheime API-sleutel blijft daardoor in de backend.

5. **Mijn werk opslaan en samenvoegen**
   Ik werkte op de branch valorant-api en sloeg mijn wijzigingen op met commits. De backendwijzigingen voegde ik samen met main. Voor de frontend maakte ik pull request #4 naar master en rondde ik de merge af. Zo staat mijn werk op GitHub en kan mijn projectgroep ermee verder.
   Afbeelding 7: de commits en de samengevoegde pull request op GitHub.

De volledige codefragmenten en grotere screenshots staan in de bijlagen, met dezelfde afbeeldingnummers.

Resultaat

Gerealiseerde functionaliteit*
"Er is voldoende inhoud van de functionaliteit gerealiseerd binnen de gestelde/geplande tijd."

Tijdens de lessen heb ik de Valorant koppeling gebouwd. Je kunt een speler opzoeken met zijn naam en tag en vervolgens zijn spelersgegevens, rank, RR en hoogste rank bekijken. De backend haalt deze gegevens op en geeft ze door aan de frontend. Ook heb ik controles en foutmeldingen toegevoegd voor als gegevens ontbreken of de aanvraag mislukt.
De belangrijkste functies van dit onderdeel zijn in deze periode gerealiseerd. Dit laat ik zien met de code en het werkende resultaat in afbeeldingen 2, 5 en 6. Mijn commits en merge in afbeelding 7 laten zien wanneer ik mijn werk heb vastgelegd en samengevoegd.

	Kwaliteit opgeleverde functionaliteit **"De opgeleverde functionaliteiten voldoen aan de eisen en wensen."** Ik ben tevreden met de backend die ik heb gemaakt. De koppeling haalt spelersgegevens, rank en RR op, zoals de bedoeling was. De code is overzichtelijk verdeeld over de service, controller en routes, zodat duidelijk is welk bestand wat doet. Ook heb ik controles en foutmeldingen toegevoegd. Een groepsgenoot gaat de frontend nog verder uitwerken, zodat de gegevens duidelijk en netjes op de website worden getoond. Mijn backendonderdeel is klaar, maar de volledige functionaliteit is nog niet helemaal af. **Kwaliteit code*** **"De kwaliteit van de code is goed."** Ik heb de code overzichtelijk verdeeld: de service haalt gegevens op bij de API, de controller controleert de invoer en verwerkt het antwoord, en de routes bepalen welke URL wordt aangeroepen. Ik gebruik duidelijke namen zoals getAccount() en getRank(), zodat mijn groepsgenoten de code makkelijk kunnen begrijpen. De API-sleutel staat apart in .env. Ook worden verbindingsproblemen opgevangen en gebruik ik één methode voor het afhandelen van API-fouten, zodat ik dezelfde code niet steeds hoef te herhalen. Dit is te zien in afbeeldingen 1, 2 en 5. **Versiebeheer*** **"Versiebeheer is effectief toegepast."** Ik heb op de aparte branch valorant-api gewerkt, zodat mijn aanpassingen niet direct op de hoofdbranch kwamen. Mijn wijzigingen heb ik opgeslagen met commits en naar GitHub gepusht. Daarna heb ik de backend samengevoegd met main en de frontend via pull request 4 met master. Hierdoor zijn mijn wijzigingen terug te vinden en kunnen mijn groepsgenoten de nieuwe code ophalen. De commits en merge zijn te zien in afbeelding 7.
Reflectie	Beschrijf welke feedback en feedforward (tips voor de toekomst) je hebt gekregen van medestudenten, collega's, stagebegeleiders, klanten, opdrachtgevers of docenten. Mijn docent was tevreden met het resultaat en vond het mooi wat ik had gemaakt. Mijn teamgenoten vonden het ook cool dat we echte Valorant-gegevens konden ophalen en gebruiken in ons project. Dat gaf mij vertrouwen in mijn werk. Ik heb hierbij geen specifieke tips voor de toekomst gekregen. Reflecteer daarna op jezelf: • Hoe beoordeel jij de kwaliteit van je eindproduct?

Ik ben tevreden met mijn eindproduct. De backend haalt de spelersgegevens, rank en RR op en de code is overzichtelijk verdeeld over verschillende bestanden. Daardoor kan mijn groepsgenoot er makkelijk mee verder voor de frontend. Ook heb ik foutmeldingen toegevoegd, zodat een mislukte aanvraag wordt opgevangen. De frontend moet nog verder worden uitgewerkt, dus het volledige onderdeel is nog niet af, maar de basis werkt.

- Tijd die er nodig was om het bewijsstuk te realiseren.

  Ik heb hier tijdens de lessen van 25 tot en met 28 september 2026 aan gewerkt. De tijd ging vooral naar het koppelen van de API, het ophalen van de juiste gegevens en het oplossen van fouten. Vooral het probleem met de API-sleutel kostte extra tijd. Het precieze aantal uren heb ik niet bijgehouden.

- Hoe verliep het proces om tot dit bewijsstuk te komen?

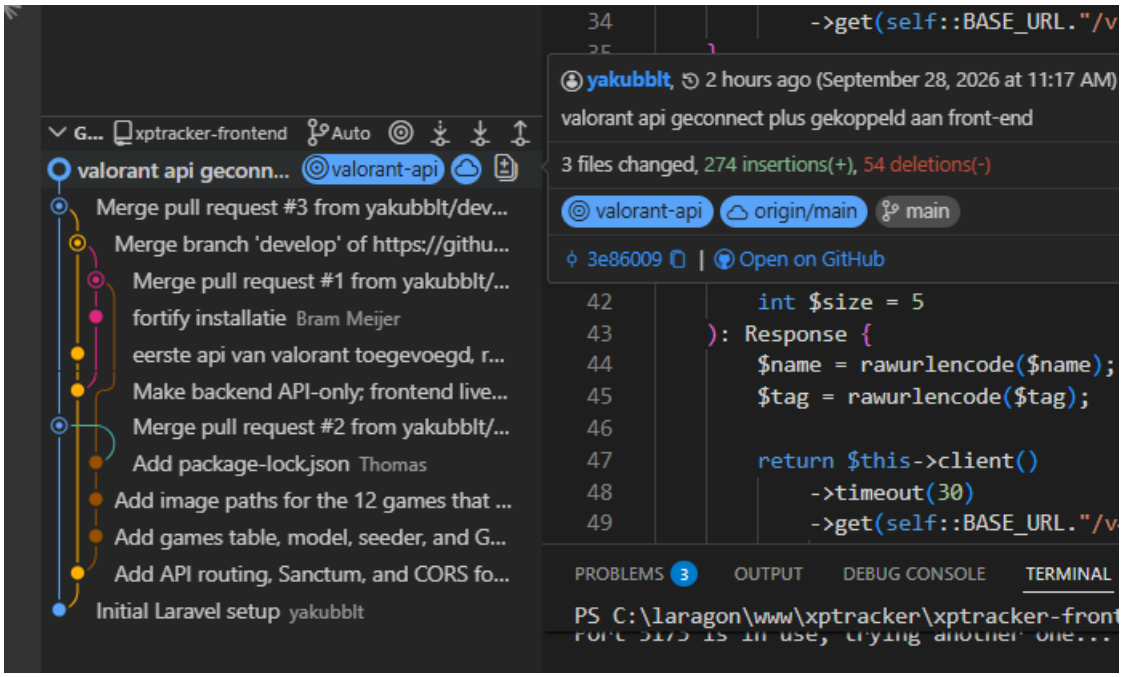
  Het proces ging niet meteen soepel. Eerst kon Laravel mijn service niet vinden en daarna werd de API-sleutel niet goed ingelezen. Door de foutmeldingen stap voor stap te bekijken, kon ik achterhalen waar het misging. Toen de verbinding werkte, kon ik verder met het ophalen van de rank en RR. Ik heb hiervan geleerd om eerst een klein onderdeel werkend te krijgen en daarna pas verder uit te breiden.

- Hoe ging de samenwerking en het overleg met anderen?

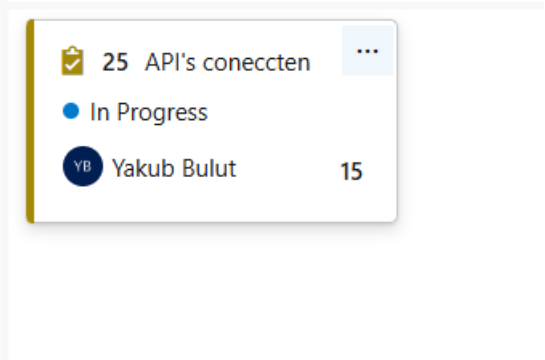
We hebben het werk verdeeld: ik werkte vooral aan de backend en de API-koppeling, en een groepsgenoot werkt de frontend verder uit. Daarbij is het belangrijk dat duidelijk is welke gegevens mijn backend teruggeeft en hoe die opgehaald kunnen worden. Mijn code staat op GitHub, zodat mijn groepsgenoot ermee verder kan. We moeten nog samen controleren of alles goed aansluit op de uiteindelijke frontend.

De volgende keer zou ik eerst controleren of de API-sleutel goed wordt ingelezen en één simpele aanvraag testen. Nu liep ik daar tijdens het bouwen tegenaan. Ook zou ik vooraf met mijn groepsgenoot afspreken welke gegevens de frontend nodig heeft en hoe ik die teruggeef. Daarna zou ik ieder onderdeel meteen testen, zodat ik fouten eerder ontdek en sneller weet waar het probleem zit.

Bijlage 1.1



Bijlage 1.2




```
    return response()->json([
        'data' => [
            'name' => $response->json('data.name'),
            'tag' => $response->json('data.tag'),
            'level' => $response->json('data.account_level'),
            'region' => $response->json('data.region'),
            'puuid' => $response->json('data.puuid'),
            'title' => $response->json('data.title'),
            'platforms' => $response->json('data.platforms', []),
            'card_image' => $response->json('data.card')
                ? 'https://media.valorant-api.com/playercards/'.$response->json('data.card').'/wideart.png'
                : null,
            'updated_at' => $response->json('data.updated_at'),
        ],
    ]);
}
```

Dit is de belangrijkste data wat word opgehaald in de API